



FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433 SOFTWARE ARCHITECTURE

PROJECT TITLE:

CLOUD NATIVE LEARNING MANAGEMENT SYSTEM (LMS)

GROUP 4

Name	Matric No.
EASHENRAJ A/L NAGARAJAN	S76773
MOHAMMAD ADAM BASHEER B. MOHD AZRUL	S75710
AMMAR HAZIQ B. SAIFUL BAHRI	S75772

PREPARED FOR:

DR MOHAMAD NOR HASSAN

Table of Content

1. PROJECT PROPOSAL.....	3
1.1 Selected Architecture Style.....	3
1.2 Problem Description.....	4
1.3 System Objectives.....	5
1.4 Initial Functional Requirements.....	6
1.5 Team Member Roles.....	7

1. PROJECT PROPOSAL

1.1 Selected Architecture Style

The architecture style that will be used in this project is the Layered Architecture Pattern. This design consists of different logical layers for an application, where each layer has a defined responsibility and communicates only with its adjacent layers. This approach of having different layers improves maintainability because components can be modified more easily, and scalability by allowing the system to scale different parts separately. These distinct layers are responsible for specific sets of functionalities. This architecture minimizes dependencies between layers, enabling greater flexibility and easier modification of individual components. For this LMS, we are using a closed layered approach, which means a layer cannot bypass another layer to access the lower layer.

The Presentation Layer is responsible for user interface design and user experience. This layer handles visual displays and gets input from users, which later communicates with the Business Layer and displays processed results.

The Business Layer is the business logic where the received information will be processed from the previous layer. This layer handles complex operations and core business rules. Having a separate layer allows modification to business rules without changing the user interface or database.

The Data Access Layer is where the data interactions happen. This layer ensures communication between the system and data storage. This layer handles CRUD operations such as create, read, update, and delete records. This layer also acts as a gatekeeper to ensure only authorized requests reach the data.

Finally, the Database Layer consists of actual database servers, which can be either relational database management systems or NoSQL databases. This layer is designed to be independent, so we can utilize a cloud database to improve reliability.

This architecture pattern allows our project to achieve Separation of Concerns, where each layer can be modified without making changes to other layers, and also allows each layer to be tested independently using test data, which helps to reduce the risk of bugs in the system.

1.2 Problem Description

Traditional Learning Management Systems (LMS) often suffer from a monolithic and tightly coupled design where the layers, such as user interface, business logic, and database, cause problems.

In traditional learning management system (LMS) designs, the user interface, business logic, and database queries are usually associated with each other. This makes system maintenance more complex. As a result, updating one feature may unintentionally break other features. This problem leads to long downtimes and high risks during system updates.

Other than that, the absence of a Data Access Layer (DAL) in a student learning management system (LMS) may cause tight coupling between the user interface, business logic, and database. Accessing features such as course materials and submitting assignments may directly interact with the database, which may cause corruption in the database. This makes the system less reliable and harder to scale.

Lastly, the traditional LMS design may create a scaling problem because the system is not modularized. Peak academic periods, such as course registration, cause traffic problems in the LMS due to the students attempting to access the database at the same time. This causes the system to crash because the platform lacks the scalability and performance to handle a high volume load.

1.3 System Objectives

The main goal of this project is to develop a scalable and maintainable Learning Management System (LMS) by applying modern architectural principles and design. The objectives are:

1. **To separate system components into distinct layers**
 - By utilizing the Layered Architecture Pattern, it can ensure that the modifications done to the user interface will not interrupt the other important parts, such as database integrity.
2. **To ensure secure data handling**
 - Establish a dedicated Data Access Layer (DAL) using Data Access Objects (DAO), protecting the system from data corruption and unauthorized access.
3. **To support scalable performance during peak usage periods**
 - Design the system structure to scale resources in response to high user demand. This specifically targets the performance drops experienced during course peak hours, such as registration or exam tests, ensuring the system remains stable despite high volumes of concurrent traffic.
4. **To reduce maintenance complexity**
 - Reduce system downtime by creating a structure where individual components can be updated, tested, and deployed independently without affecting the other components.
5. **To provide high performance**
 - Leverage cloud environments to ensure the LMS is accessible 24/7, providing a responsive experience for both students and lecturers to access the materials.

1.4 Initial Functional Requirements

To effectively demonstrate a layered, cloud-native architecture, the system's functional requirements are categorized into several modules. This ensures the separation of concerns across the presentation, business logic, and data access layers.

1. User Identity & Access Management Module

- a. User Registration: The system shall allow new users to create an account by providing basic details such as name, email, password, and roles.
- b. Authentication: The system shall securely authenticate users upon login and generate a session token to manage access across different layers of the application.
- c. Role-Based Access Control: The system shall enforce role-specific permissions, for example, by preventing a Student from accessing a course-creation API endpoint.
- d. Profile Management: Users shall be able to view and update their personal profile information and change their passwords.

2. Course Management Module

- a. Course Creation: The system shall allow instructors to create a new course by defining a title, description, syllabus, and enrollment capacity.
- b. Course Modification: Instructors shall be able to update or delete courses they have created.
- c. Course Catalog: The system shall display a catalog of all available courses, allowing Students to search or filter by course name or instructor.
- d. Enrollment Processing: The system shall allow Students to enroll in available courses and automatically update their personalized dashboard to reflect active enrollments.

3. Content Delivery and Management Module

- a. Material Upload: The system shall allow Instructors to upload educational materials such as PDF documents, slide decks, or links to external resources to specific courses.
- b. Content Organization: Instructors shall be able to organize uploaded materials into modules or weeks for structured learning.
- c. Material Access: The system shall allow enrolled Students to view, access, and download course materials from the frontend interface.

4. Assignment and Assessment Module
 - a. Assignment Creation: Instructors shall be able to create assignments, specifying the title, instructions, maximum score, and a strict submission deadline.
 - b. File Submission: The system shall allow Students to upload file submissions for active assignments before the specified deadline.
 - c. Submission Tracking: The system shall track and display the submission's status, such as “Submitted”, “Late”, “Pending”, or “Overdue” on the Student's dashboard.
 - d. Grading and Feedback: Instructors shall be able to view student submissions, assign a numerical grade, and provide text-based feedback, which the system will make visible to the respective Student.
5. System & Architectural Requirements
 - a. API Communication: The system's frontend layer shall communicate with the backend business logic exclusively through well-defined RESTful API endpoints.
 - b. Data Persistence: The system shall accurately store and retrieve all user, course, and assignment data using a designated database.

1.5 Team Member Roles

Name	Role
EASHENRAJ A/L NAGARAJAN	DATABASE ARCHITECT
MOHAMMAD ADAM BASHEER B. MOHD AZRUL	FRONTEND & USER EXPERIENCE (UX) LEAD
AMMAR HAZIQ B. SAIFUL BAHRI	BACKEND LEAD

References

1. Cherif, Y. (2024, November 28). *Understanding the Layered Architecture Pattern: A Comprehensive guide*. DEV Community.
https://dev.to/yasmine_ddec94f4d4/understanding-the-layered-architecture-pattern-a-comprehensive-guide-1e2j
2. *Home | Moodle.org*. (n.d.). <https://moodle.org/>
3. GeeksforGeeks. (2025, July 23). *Layered architecture of cloud*. GeeksforGeeks.
<https://www.geeksforgeeks.org/devops/layered-architecture-of-cloud/>
4. Savolainen, J., & Myllarniemi, V. (2009, September). Layered architecture revisited—Comparison of research and practice. In *2009 Joint Working IEEE/IFIP Conference on Software Architecture & European Conference on Software Architecture* (pp. 317-320). IEEE.
5. Aldheleai, H. F., Bokhari, M. U., & Alammari, A. (2017). Overview of cloud-based learning management system. *International Journal of Computer Applications*, 162(11), 41-46.